Atomic ինչ է — Խորը ուղեցույց

C++ Concurrency presentation

Այս ուղեցույցը մանրամասն բացատրում է `std::atomic`-ը՝ inch e, inchpes e lucum race-y aŕanc mutex-i, atomic vs mutex, operacianer, memory order, orinaknerov ev interview harcerov.

---

1. Ի՞նչ է Atomic-ը

`std::atomic`-y tesak e, vori operacianery **anbajaneli (atomic)** en -- mek anbajaneli qayl, vory mej ayl thread chi karox mijamtel. Chka race condition aŕanc mutex-i.

```
#include <atomic>
std::atomic<int> counter{0};

void increment() {
    counter++;    // ATOMIC -- read-modify-write mek anbajaneli qayl
}
// 2 thread -> counter chisht e, aŕanc mutex-i
```

---

2. Xndiry — inchu sovorakan int-y race e

```
int counter = 0;      // sovorakan
counter++;            // 3 qayl: read, +1, write -> thread-ner xaŕnvum
```

```
std::atomic<int> counter{0};   // atomic
counter++;            // MEK anbajaneli qayl -> chka race
```

Atomic-y erashxavorum e, vor `counter++`-y katarvi amboxjovin kam ochinch (chka mijanki vichak)։

---

3. Atomic operacianer

```
std::atomic<int> a{0};

a++;  ++a;          // atomic increment
a--;  --a;          // atomic decrement
a += 5;             // atomic add
a.load();           // atomic read
a.store(10);        // atomic write
a.exchange(20);     // set nor, veradarcnum hin
int expected = 20;
a.compare_exchange_strong(expected, 30);  // CAS (compare-and-swap)
```

compare_exchange (CAS) — lock-free-i himq

```
// ete a == expected -> a = desired, return true
// hakaŕak depqum -> expected = a, return false
std::atomic<int> a{5};
int expected = 5;
a.compare_exchange_strong(expected, 10);   // a darav 10
```

CAS-y lock-free algoritmneri himqn e.

---

4. Atomic vs Mutex

```
ATOMIC:
  + Aveli arag (lock-free, chka blocking)
  + Poqr operacia-i hamar (counter, flag, pointer)
  - Miayn PAŔZ operacia (mek uzhamary)
  - Bardz logic-i hamar chi ashxatum

MUTEX:
  + Bardz critical section (mi qani uzhamar, bardz logic)
  - Aveli dandaghn (blocking, context switch)
```

	atomic	mutex
Aragutyun	arag (lock-free)	dandaghn (blocking)
Kiraŕum	poqr operacia (counter/flag)	bardz critical section
Blocking	voch	ayo
Bardz logic	voch	ayo

Kanon. poqr (counter, flag, pointer) -> atomic; bardz (mi qani uzhamar, container poxum) -> mutex.

---

5. Real orinak

Atomic counter

```
std::atomic<int> counter{0};
void work() {
    for (int i = 0; i < 100000; i++)
        counter++;   // atomic, aŕanc mutex, arag
}
```

Atomic flag (stop signal)

```
std::atomic<bool> stop{false};

void worker() {
    while (!stop.load())    // atomic read
        doWork();
}
void main_control() {
    stop.store(true);       // atomic write -> worker kangnum
}
```

---

6. Memory Order (aŕajacvac)

```
a.store(10, std::memory_order_relaxed);   // ochmi karg erashxiq (arag)
a.load(std::memory_order_acquire);        // acquire
a.store(10, std::memory_order_release);   // release
// default: memory_order_seq_cst (amenaxist, amenaanvtang)
```

Junior mak. default (`seq_cst`) bavakan e; memory order-y optimization e aŕajacvac depqerum.

---

7. is_lock_free

```
std::atomic<int> a;
cout << a.is_lock_free();   // sovorabar true poqr tiperi hamar
```

Poqr tiper (int, pointer) sovorabar lock-free en (uxix CPU instruction); mec object-ner karox en ners mutex ogtagorcel.

---

8. Interview harcer & patasxanner

atomic inch e?

> *Tesak vori operacianery anbajaneli en; race condition aŕanc mutex-i poqr operacia-neri hamar.*

Inchu sovorakan int-y race e, atomic-y voch?

> *int++ 3 qayl e (read/+1/write); atomic++-y mek anbajaneli qayl.*

atomic vs mutex?

> *atomic arag (lock-free) poqr operacia-i; mutex bardz critical section-i (blocking). Poqr -> atomic, bardz -> mutex.*

compare_exchange (CAS) inch e?

> *Ete arjeqy == expected, poxum desired-ov (atomic); lock-free algoritmneri himq.*

Memory order inch e?

> *Atomic operacia-neri karganavorman erashxiqner; default seq_cst (amenaxist).*

is_lock_free?

> *atomic-y iskapes lock-free e te ners mutex ogtagorcum (poqr tiper -> lock-free).*

atomic flag inchi hamar?

> *Stop/ready signal thread-neri mijev, aŕanc mutex.*

Cheat Sheet

```
ATOMIC = anbajaneli operacia -> race aŕanc mutex (poqr operacia-i hamar)

  a++ / a.load() / a.store(x) / a.exchange(x)
  a.compare_exchange_strong(exp, des)  -> CAS (lock-free himq)

ATOMIC vs MUTEX:
  atomic -> arag (lock-free), poqr operacia (counter/flag/pointer)
  mutex  -> dandaghn (blocking), bardz critical section

MEMORY ORDER: default seq_cst (xist) | relaxed/acquire/release (optimization)
is_lock_free() -> poqr tiper sovorabar lock-free

KANON: counter/flag -> atomic | bardz logic/container -> mutex
```